

Anatomie du moteur

Du dataset à la sortie d'une clause, puis à un round de boosting
Un seul algorithme, appliqué identiquement à Iris, Digits, XOR et MNIST

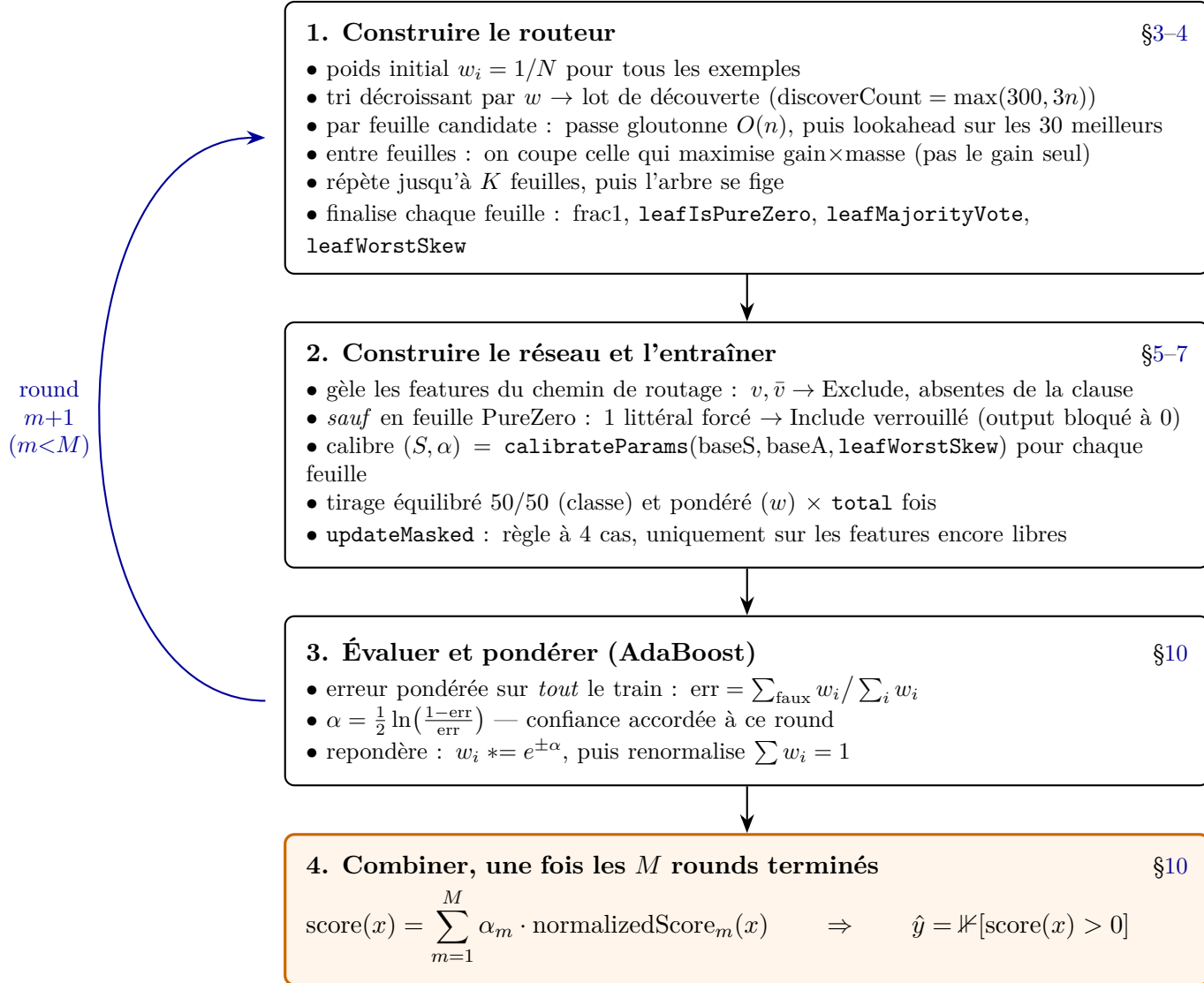


Figure 1: Vue d'ensemble complète du mécanisme, tous les niveaux inclus : construction du routeur (deux passes + sélection par masse), construction et calibration du réseau, entraînement, évaluation/repondération AdaBoost, et combinaison finale des M réseaux. Chaque puce renvoie à la section qui la détaille avec l'exemple fil rouge chiffré.

Contents

1	L'exemple fil rouge	3
2	Vue d'ensemble : le trajet d'un exemple, du dataset à la sortie	3
2.1	Rappel : que veut dire "littéral inclus" ?	4
3	Construire le routeur	4
3.1	L'impureté de Gini pondérée	5
3.2	L'algorithme, une feuille à la fois	5
3.3	Pourquoi deux passes : glouton, puis lookahead limité	5
3.4	Choisir la feuille à couper : gain $\times$ masse	6
4	Statuer sur chaque feuille finale	6
5	Calibrer S et α contre les features déséquilibrées	8
5.1	D'où vient le seuil B	8
6	Geler les features du routeur, verrouiller les feuilles PureZero	8
6.1	Cas standard : exclure les deux littéraux	9
6.2	Cas PureZero : verrouiller la sortie à 0	9
7	Mettre à jour une clause : la règle à 4 cas, masquée	10
7.1	Comment la clause répond ensuite : les trois cas de <code>score()</code>	10
8	Entraîner : tirage équilibré, total mises à jour	10
9	Mesurer l'erreur, pondérer (AdaBoost)	11
10	Combiner les M réseaux et décider	12
11	Où tout ceci s'inscrit : le round complet	12

1 L'exemple fil rouge

Pour rendre chaque étape concrète, on utilise partout dans ce document le même petit dataset jouet : $n = 4$ features booléennes $x = (x_0, x_1, x_2, x_3)$, une étiquette $y \in \{0, 1\}$, et un routeur déjà construit à $K = 3$ feuilles par deux coupures successives :

- la racine coupe sur x_0 : à gauche ($x_0=0$) → **feuille A** ;
- à droite ($x_0=1$), une deuxième coupure sur x_2 donne **feuille B** ($x_2=0$) et **feuille C** ($x_2=1$).

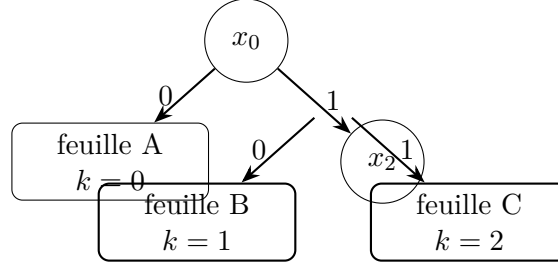


Figure 2: Le routeur jouet utilisé dans tout ce document : 2 coupures, 3 feuilles. Feuille A : $x_0=0$ fixé (1 feature gelée). Feuille B et C : $x_0=1, x_2 \in \{0, 1\}$ fixés (2 features gelées), x_1, x_3 restent libres.

Trois faits, calculés une fois pour toutes sur les exemples d’entraînement tombés dans chaque feuille, complètent ce routeur (section 4 pour le détail) :

	feuille A ($k=0$)	feuille B ($k=1$)	feuille C ($k=2$)
features gelées	$\{0\}$, fixée à 0	$\{0, 2\}$, fixées à (1, 0)	$\{0, 2\}$, fixées à (1, 1)
frac1 pondérée	0.03	0.52	0.18
leafIsPureZero	vrai	faux	faux

La feuille A est donc *presque toujours négative* (PureZero) — on y reviendra en détail à la section 6.

2 Vue d’ensemble : le trajet d’un exemple, du dataset à la sortie

Avant de détailler la construction, voici tout le trajet d’un exemple à travers un réseau *déjà construit* — ce que fait `MultiClauseNetwork::output(x)`. Un réseau, c’est : un routeur figé (section 3) qui aiguille chaque exemple vers l’une des K clauses, et K clauses, chacune un simple **ET logique** de littéraux dont les automates ont été appris (section 7).

Exemple fil rouge 2.1. On prend $x = (1, 1, 1, 0)$. Le routeur teste $x_0=1$ (va à droite), puis $x_2=1$ (va à droite) : x atterrit en **feuille C** ($k=2$). Supposons que la clause de cette feuille ait appris, sur ses deux features libres $\{1, 3\}$: $v[1] = \text{Include}$ (le littéral “ x_1 ”) et $\bar{v}[3] = \text{Include}$ (le littéral “ $\neg x_3$ ”), tout le reste exclu.

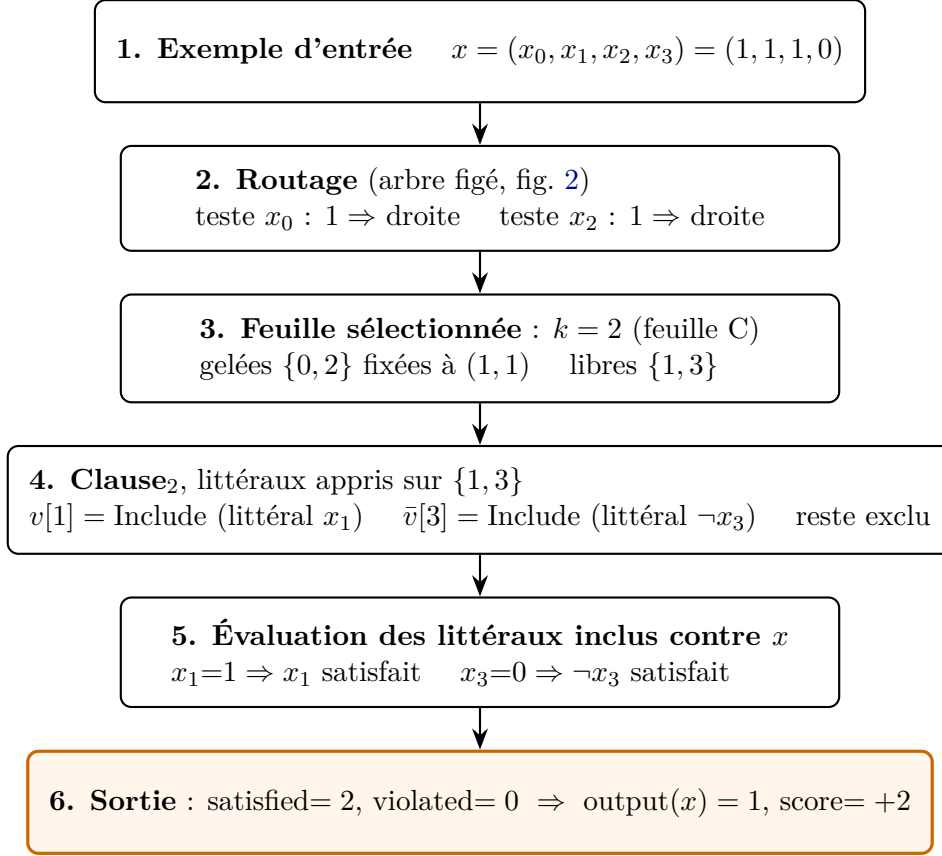


Figure 3: Le trajet complet d’un exemple, du dataset à la sortie d’une clause. Chaque étape est développée dans une section dédiée : le routage (sections 3–4), le gel des features (section 6), et l’évaluation/l’apprentissage de la clause (section 7).

2.1 Rappel : que veut dire “littéral inclus” ?

Chaque feature i possède deux automates : $v[i]$ (associé au littéral x_i) et $\bar{v}[i]$ (associé au littéral $\neg x_i$). Chacun a un état entier dans $\{1, \dots, 2N\}$: au-dessus de N , le littéral est Include (il compte dans la clause) ; à N ou en-dessous, il est Exclude (il est ignoré). La clause est le ET logique de tous les littéraux inclus. Elle est **violée** dès qu’un seul littéral inclus est faux pour x ; elle est **satisfaite** seulement si *tous* ses littéraux inclus sont vrais pour x (ou si elle n’en a aucun).

3 Construire le routeur

Le routeur est construit *une fois par round de boosting* par `buildWeightedRouterTree`, sur un lot pondéré d’exemples. Son rôle : couper l’espace des exemples en K groupes (les feuilles), aussi purs que possible vis-à-vis de y , en ajoutant une coupure à la fois.

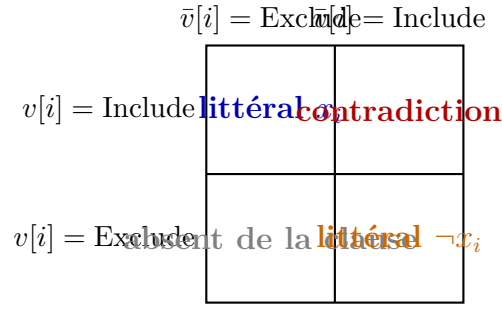


Figure 4: Les quatre configurations possibles de $(v[i], \bar{v}[i])$ pour une seule feature i (notation identique à *convergence_sans_bruit.tex*). Le cas “absent de la clause” ($v, \bar{v}$ tous deux Exclude) est celui utilisé pour toute feature gelée par le routeur (section 6).

3.1 L’impureté de Gini pondérée

Pour juger si une coupure est bonne, on mesure l’impureté de Gini *pondérée* d’un ensemble d’exemples E :

$$\text{wSum} = \sum_{e \in E} w_e, \quad \text{wPos} = \sum_{e \in E, y_e=1} w_e, \quad p_1 = \frac{\text{wPos}}{\text{wSum}}, \quad \text{Gini}(E) = 2p_1(1-p_1).$$

$\text{Gini}(E) = 0$ si E est pur (tous y_e identiques), $\text{Gini}(E) = 0.5$ si E est parfaitement mélangé (50/50). Couper E en E_L, E_R sur une feature f donne un **gain** :

$$\text{gain}(f) = \text{Gini}(E) - \underbrace{\left[\frac{\text{wSum}(E_L)}{\text{wSum}(E)} \text{Gini}(E_L) + \frac{\text{wSum}(E_R)}{\text{wSum}(E)} \text{Gini}(E_R) \right]}_{\text{impureté pondérée après coupure}}.$$

3.2 L’algorithme, une feuille à la fois

L’arbre part d’une seule feuille (la racine, qui contient tous les exemples du lot). Tant qu’il y a moins de K feuilles :

1. pour **chaque feuille actuelle**, on cherche sa meilleure coupure possible (feature + gain), avec la méthode en 2 passes ci-dessous ;
2. parmi toutes les feuilles, on choisit celle dont la coupure a le plus grand **gain** $\times$ **masse de la feuille** (section 3.4) ;
3. on coupe cette feuille en deux nouvelles feuilles, on retire l’ancienne, et on recommence.

3.3 Pourquoi deux passes : glouton, puis lookahead limité

Une passe purement gloutonne (tester chaque feature seule, garder la meilleure) suffit dans la grande majorité des cas, mais échoue sur des structures type XOR. Exemple minimal à 2 features signal x_0, x_1 avec $y = x_0 \oplus x_1$:

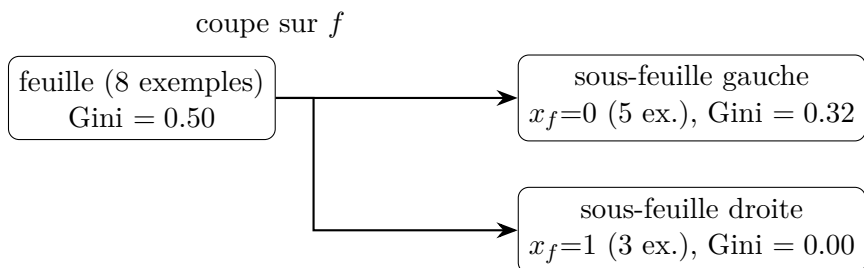


Figure 5: Une itération, en substance : une feuille se coupe en deux sur la feature choisie. Le gain de cette coupure précise serait ici $0.50 - (\frac{5}{8} \cdot 0.32 + \frac{3}{8} \cdot 0.00) = 0.30$. (Valeurs illustratives.)

x_0	x_1	y
0	0	0
0	1	1
1	0	1
1	1	0

Couper sur x_0 seul donne deux enfants à 50/50 ($x_0=0$ contient $y \in \{0, 1\}$ à parts égales, pareil pour $x_0=1$) : **gain nul**. Même chose pour x_1 seul. Le signal n'existe que dans la *combinaison* des deux features — une passe gloutonne ne voit donc, à tort, aucune raison de couper ici.

Le **lookahead** corrige ce problème : pour un candidat f , on simule un *deuxième* niveau de coupure sur chaque enfant (avec la meilleure feature restante), et on mesure le gain à deux coups d'avance plutôt qu'à un seul. Sur l'exemple ci-dessus, couper sur x_0 puis, dans chaque enfant, sur x_1 , sépare parfaitement y — le lookahead voit ce gain, la passe gloutonne ne le voit pas.

Le problème : simuler le lookahead sur les n features coûterait $O(n^2)$, intraitable pour MNIST ($n = 784$). La solution retenue, **identique pour tous les datasets** : classer d'abord toutes les features par gain glouton ($O(n)$), puis ne raffiner par lookahead que les LOOKAHEAD_TOPM = 30 meilleures ($O(30 \cdot n)$). Pour Iris ($n=16$) et XOR ($n=12$), $n < 30$ donc *toutes* les features sont raffinées — le comportement est celui du lookahead complet. Pour Digits ($n=192$) et MNIST ($n=784$), seules les 30 meilleures le sont, ce qui suffit en pratique.

3.4 Choisir la feuille à couper : gain $\times$ masse

Une fois la meilleure coupure connue pour chaque feuille, laquelle couper en premier ? Le code compare `leafBestGain * leafMass`, où `leafMass` est la somme des poids des exemples de la feuille — **pas** le gain seul.

4 Statuer sur chaque feuille finale

Une fois l'arbre figé (K feuilles, ou moins si le lookahead ne trouve plus de coupure utile), chaque feuille reçoit quatre étiquettes, calculées **une seule fois** à partir des exemples pondérés qui y sont tombés.

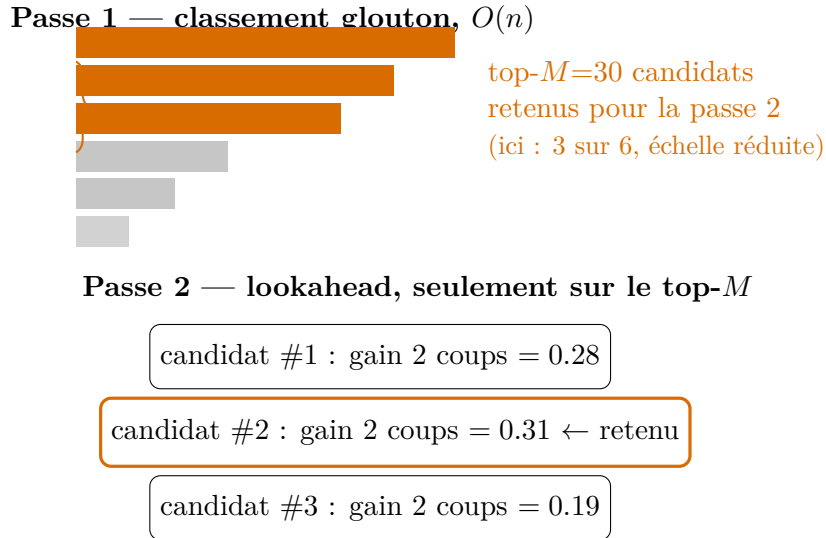


Figure 6: Le classement glouton trie *toutes* les features sans regarder plus loin. Seuls les LOOKAHEAD_TOPM meilleurs reçoivent la simulation d’un second niveau de coupure (LookaheadBuffers) — ce qui peut changer le gagnant (le candidat #2, moins bon au premier coup d’œil, l’emporte ici une fois le deuxième niveau simulé).

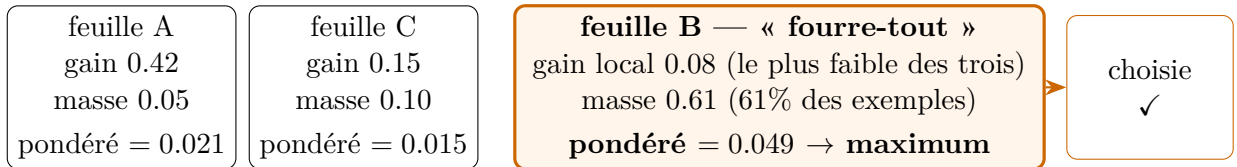


Figure 7: Sans ce facteur de masse, une feuille géante mal triée (typique de MNIST, où une immense feuille “fourre-tout” concentrerait 53 à 89% des négatifs) n’était *jamais* subdivisée : son gain *local* semblait toujours pire que celui de petites feuilles déjà quasi pures, alors qu’elle concernait bien plus d’exemples au total. Ce correctif de pondération par la masse est celui qui a fait progresser le plus l’accuracy sur MNIST.

frac1 la proportion pondérée d’exemples $y=1$: $\text{frac1} = \text{wPos}/\text{wSum}$ sur les exemples de la feuille.

leafIsPureZero vrai si $\text{frac1} \leq \text{noiseTol}$ — la feuille est jugée presque toujours négative, même en tolérant un peu de bruit d’étiquette.

leafMajorityVote 1 si $\text{frac1} \geq 0.5$, sinon 0 — sert uniquement à la baseline diagnostique “routeur seul”, jamais à la prédiction du système complet.

leafWorstSkew on parcourt toutes les features *encore libres* f (non gelées par le chemin), on calcule leur proportion pondérée $q_f = P(x_f=1)$ dans la feuille, et on garde $\min_f \min(q_f, 1 - q_f)$ — la feature libre la plus déséquilibrée, c’est-à-dire la plus dangereuse pour l’apprentissage (section 5).

Exemple fil rouge 4.1. Pour la feuille C ($k=2$), features libres $\{1, 3\}$: si x_1 apparaît à $q_1 = 0.47$ (presque équilibrée) et x_3 à $q_3 = 0.06$ (très déséquilibrée), alors $\text{leafWorstSkew} = \min(0.47, 0.53, 0.06, 0.94) = 0.06$. C’est ce chiffre, et non 0.47, qui pilotera la calibration : le pire cas gouverne.

	feuille A	feuille B	feuille C
frac1 pondérée	0.03	0.52	0.18
leafIsPureZero	vrai	faux	faux
leafMajorityVote	0	1	0
leafWorstSkew	0.31	0.44	0.06

5 Calibrer S et α contre les features déséquilibrées

Un automate de Tsetlin standard suppose implicitement que ses deux issues ($x_i=0$ et $x_i=1$) sont à peu près équiprobables. Si une feature libre est en réalité très déséquilibrée (ex. $q = 0.06$, feuille C ci-dessus) — sans être totalement constante — l’automate peut dériver vers Include à tort, uniquement parce qu’il voit rarement le cas contraire, pas parce que la feature est informative. `calibrateParams(baseS, baseA, worstSkew)` réduit S pour neutraliser ce risque.

5.1 D’où vient le seuil B

On se place dans le pire cas : une feature libre de skew $q = \min(q_i, 1 - q_i)$, totalement indépendante de y (bruit pur), avec y équilibré à 50/50 (garanti par l’échantillonnage équilibré, section 8). Les probabilités de transition de $v[i]$ deviennent (cf. le tableau à 4 cas, section 7) :

$$p_v^+ = (1 - q) \cdot \frac{1}{2} \cdot S, \quad p_v^- = (1 - q) \cdot \frac{1}{2} \cdot \alpha + q \cdot \frac{1}{2} \cdot S.$$

L’automate reste, à tort, poussé vers Include tant que $p_v^+ > p_v^-$, soit $S/\alpha > (1 - q)/(1 - 2q) =: B(q)$. $B(q)$ est donc le ratio S/α **au-delà duquel** une feature purement bruitée peut dériver vers Include. `margin=4` impose une marge de sécurité : on veut rester *sous* $B(q)/4$, pas juste sous $B(q)$.

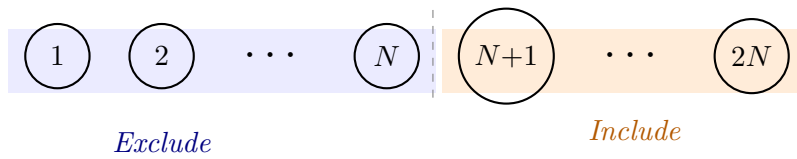


Figure 8: L’état d’un automate se lit sur une seule droite : au-dessus de N , Include; à N ou en-dessous, Exclude.

Concrètement, quand $q < 0.40$: `calibrateParams` fixe $\alpha=1$ et $S = (B(q)/4) \cdot \alpha$, ce qui ramène exactement le ratio S/α à la limite de sécurité $B(q)/4$ — ni plus prudent que nécessaire, ni moins.

6 Geler les features du routeur, verrouiller les feuilles PureZero

Une clause ne doit **pas** réapprendre ce que le routeur a déjà décidé : à l’intérieur d’une feuille donnée, une feature gelée est *constante* (par construction, tous les exemples de

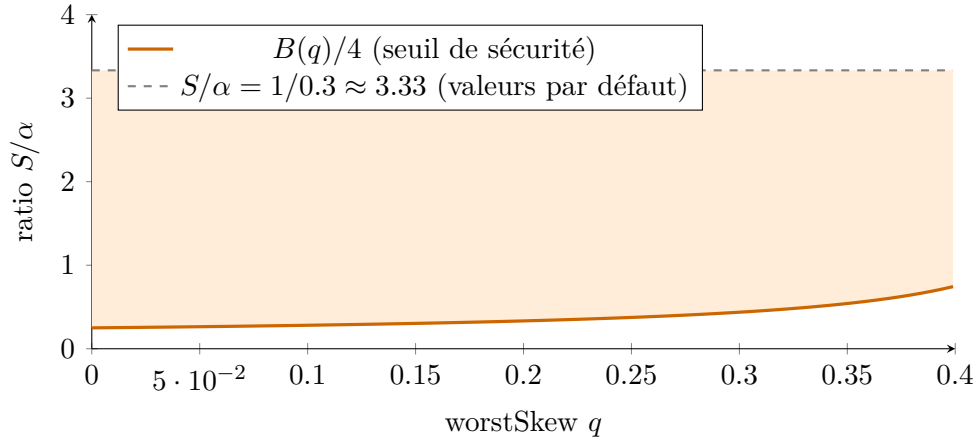


Figure 9: Pour $S=1, \alpha=0.3$ (défaut), le ratio 3.33 reste toujours au-dessus de $B(q)/4$ tant que $q < 0.40$ (zone orange) : la calibration **s’active systématiquement** pour toute feature libre avec leafWorstSkew < 0.40 . Elle ne garde les valeurs par défaut que si la feuille n’a plus de feature libre vraiment déséquilibrée ($q \geq 0.40$).

la feuille C ont $x_0=1$). Un automate confronté à une feature constante ne voit jamais qu’une seule des deux lignes du tableau à 4 cas : il dérive dans une seule direction, sans aucune force contraire pour l’arrêter — ce n’est pas un problème d’apprentissage lent, c’est une dérive garantie. La solution : ne pas apprendre du tout sur ces features, et fixer leurs deux automates une fois pour toutes à la construction du réseau.

6.1 Cas standard : exclure les deux littéraux

Pour une feature gelée i , si la feuille n’est **pas** PureZero, le code fixe simplement $v[i].state = 1$ et $\bar{v}[i].state = 1$: les deux littéraux sont Exclude, la feature **n’apparaît plus du tout** dans la clause (cadran “absent de la clause” de la figure 4). C’est le cas de $\{0, 2\}$ pour la feuille C dans l’exemple fil rouge (section 11).

6.2 Cas PureZero : verrouiller la sortie à 0

Pour la feuille A (PureZero, frac1 = 0.03), le code exploite en plus ce qu’on sait déjà : cette feuille est presque toujours négative. Plutôt que de laisser la clause l’apprendre (ce qui gaspillerait un littéral et prendrait du temps), on **câble en dur** un unique littéral garanti faux.

Exemple fil rouge 6.1. En feuille A, x_0 est fixée à 0. Puisque leafFixedValue = 0, le code force $v[0].state = 2N$ (Include) et $\bar{v}[0].state = 1$ (Exclude). Le littéral inclus est donc “ x_0 ” — or $x_0 \equiv 0$ dans cette feuille, donc ce littéral est **toujours violé**. Dès qu’un littéral inclus est violé, la clause entière est violée (section 7.1) : **output(x) = 0 pour tout x de cette feuille**, sans qu’aucun automate n’ait eu besoin d’apprendre quoi que ce soit.

feature libre ($\{1, 3\}$ en C) $v, \bar{v}$ non fixés apprennent normalement (section 7)	feature gelée, standard ($\{0, 2\}$ en feuille B ou C) $v[i] = \text{Exclude}$, $\bar{v}[i] = \text{Exclude}$ absente de la clause	feature gelée, PureZero (la 1 ^{re} de la feuille A) $v[0] = \text{Include (state=2N)}$ $\bar{v}[0] = \text{Exclude}$ <i>littéral toujours violé</i>
--	--	---

Figure 10: Les trois traitements possibles d’une feature à la construction du réseau. Un seul littéral forcé par feuille suffit : `score()` retourne “violé” dès que *n’importe quel* littéral inclus est faux (section 7.1).

7 Mettre à jour une clause : la règle à 4 cas, masquée

En dehors des features gelées, une clause apprend exactement la règle à 4 cas démontrée dans *convergence_sans_bruit.tex* — `updateMasked` se contente de sauter les features où `active[i]=false`.

$x[i]$	y	Action sur $v[i]$ (littéral x_i)	Action sur $\bar{v}[i]$ (littéral $\neg x_i$)
0	0	<code>towardInclude(S)</code>	<code>towardExclude(S)</code>
0	1	<code>towardExclude(α)</code>	rien
1	0	<code>towardExclude(S)</code>	<code>towardInclude(S)</code>
1	1	rien	<code>towardExclude(α)</code>

Exemple fil rouge 7.1. Reprenons $x = (1, 1, 1, 0)$ routé en feuille C, avec $y = 1$ (vrai label de cet exemple). Seules les features libres $\{1, 3\}$ sont mises à jour :

- $i = 1$: $x_1=1, y=1 \Rightarrow$ rien sur $v[1]$, et $\bar{v}[1]$ fait `towardExclude( $\alpha$ )` ;
- $i = 3$: $x_3=0, y=1 \Rightarrow v[3]$ fait `towardExclude( $\alpha$ )`, rien sur $\bar{v}[3]$.

$v[0], \bar{v}[0], v[2], \bar{v}[2]$ (gelées) ne bougent pas.

7.1 Comment la clause répond ensuite : les trois cas de `score()`

Cas	Condition	<code>score()</code>	Sens
1	aucun littéral inclus	0	clause vide, neutre
2	≥ 1 littéral inclus violé	−violated	confiance que ce n’est pas cette classe
3	tous les littéraux inclus satisfaits	+satisfied	confiance que c’est cette classe

8 Entraîner : tirage équilibré, total mises à jour

`WeightedSampler` garantit que chaque mise à jour voit la classe 0 et la classe 1 avec une probabilité 50/50 — même si le dataset réel est déséquilibré — tout en respectant, à l’intérieur de chaque classe, les poids du boosting (les exemples que le round précédent a mal classés sont tirés plus souvent).

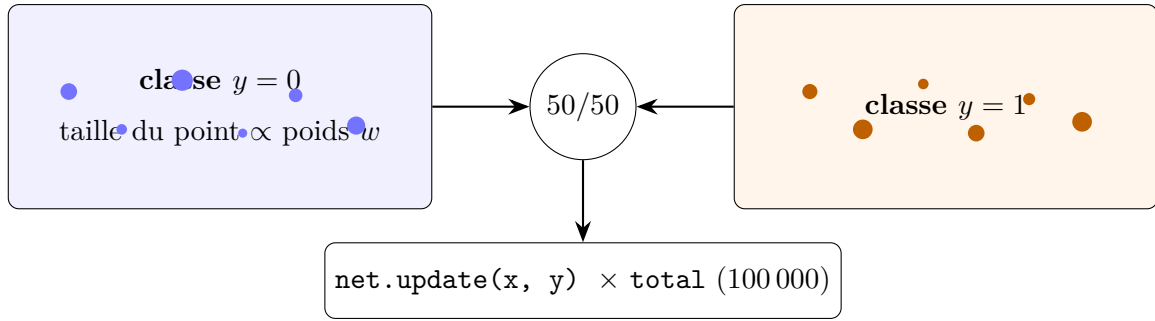


Figure 11: À chaque tirage : pile ou face 50/50 pour choisir la classe, puis tirage pondéré (`discrete_distribution`) par poids w à l'intérieur de cette classe. C'est ce rééquilibrage systématique qui rend valide l'hypothèse “ y à 50/50” utilisée dans la dérivation de `calibrateParams` (section 5).

9 Mesurer l'erreur, pondérer (AdaBoost)

Une fois les `total` mises à jour terminées, le réseau du round est évalué sur **tout** l'ensemble d'entraînement (pas seulement le lot de découverte), pondéré par les poids courants w :

$$\text{err} = \frac{\sum_i w_i}{\sum_i w_i} \in [10^{-6}, 0.999], \quad \alpha = \frac{1}{2} \ln \left(\frac{1 - \text{err}}{\text{err}} \right).$$

α est grand si err est petit (le round est fiable, on lui donne une forte voix au moment de combiner, section 10) et proche de 0 si $\text{err} \approx 0.5$ (le round n'a fait guère mieux que le hasard). Ensuite, chaque poids est mis à jour puis renormalisé :

$$w_i \ast = \begin{cases} e^\alpha & \text{si } i \text{ mal classé} \\ e^{-\alpha} & \text{sinon} \end{cases} \quad \text{puis} \quad w_i /= \sum_i w_i.$$

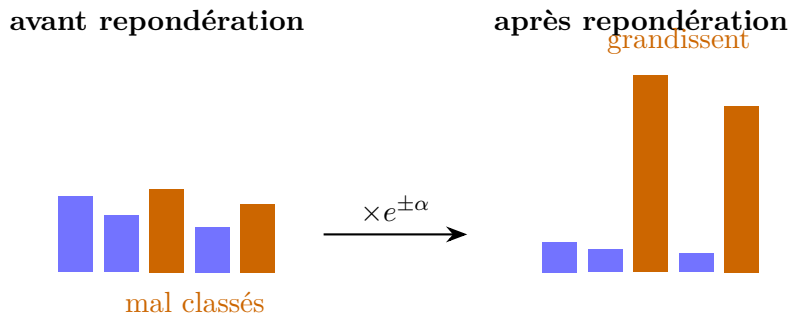


Figure 12: Les exemples mal classés voient leur poids croître, les bien classés décroître (puis tout est renormalisé). Le *prochain discoverBatch* (trié par w , section 3) sera donc dominé par les exemples encore mal classés : le round suivant est forcé de router et d'apprendre spécifiquement là où le round précédent a échoué.

10 Combiner les M réseaux et décider

Après les M rounds, chaque réseau vote avec son `score()`, normalisé par le nombre de littéraux de la clause utilisée ($\text{normalizedScore} = \text{score}/n_{\text{lit}}$, pour rendre les votes comparables entre clauses de tailles différentes), pondéré par la confiance α_m que le boosting lui a accordée :

$$\text{score}(x) = \sum_{m=1}^M \alpha_m \cdot \text{normalizedScore}_m(x), \quad \hat{y} = \begin{cases} 1 & \text{si } \text{score}(x) > 0 \\ 0 & \text{sinon.} \end{cases}$$

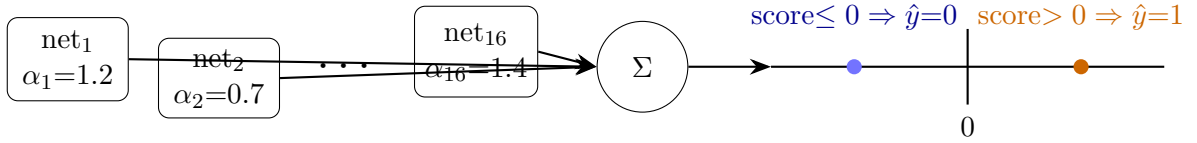


Figure 13: Vote pondéré final : un round fiable (grand α) pèse fort, un round peu fiable pèse peu — le mécanisme AdaBoost standard, appliqué ici à des réseaux routeur+clauses plutôt qu'à des souches de décision classiques.

Note 10.1. En parallèle de $\text{score}(x)$, chaque fichier de validation calcule aussi une *baseline* $\sum_m \alpha_m (\text{leafMajorityPurity}_m[\text{leaf}] - 0.5)$ — le même vote pondéré, mais en remplaçant la clause par la simple pureté de la feuille où l'exemple atterrit. Comparer les deux accuracies (ex. XOR : 100.0% avec clauses contre 50.1% sans) est la preuve empirique que ce sont bien les **clauses**, et non le seul routeur, qui portent la prédiction.

11 Où tout ceci s'inscrit : le round complet

Les sections précédentes détaillent *une passe d'inférence* (comment un réseau déjà construit répond à un exemple) et les mécanismes de construction (routeur, gel, calibration). Ensemble, ils forment un round de `trainBoostedClass`, répété M fois :

Portée : `trainBoostedClass · buildWeightedRouterTree · calibrateParams · MultiClauseNetwork`
— tous dans `tsetlin_engine.h`. Appliqué identiquement à Iris, Digits, XOR, MNIST.

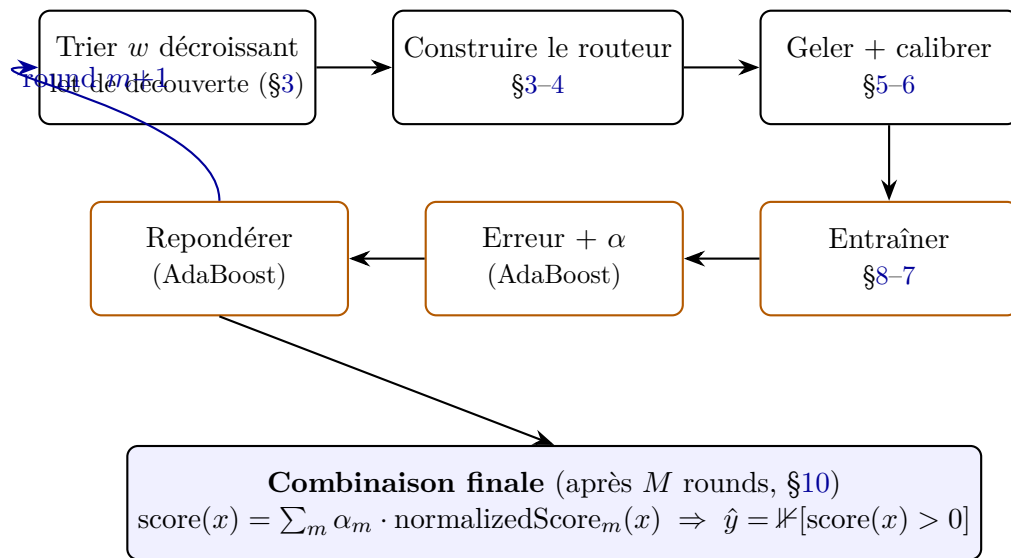


Figure 14: Le round de boosting complet. Ce schéma est identique pour les quatre datasets validés — seuls n , K , M et `noiseTol` changent d’un fichier à l’autre.